MODULE 4

SOUNDS & INTERRUPTS IN ARDUINO

Sound generation is an interesting feature of Arduino projects. By using a speaker or piezo buzzer, Arduino can produce tones, alarms, or even simple music. This is achieved by generating electrical signals with different frequencies that drive a speaker to vibrate and produce sound. The Arduino provides a built-in function called tone( ) which simplifies sound generation.

Sound is a **vibration that travels through a medium such as air**. When these vibrations reach the human ear, they are perceived as sound.

Properties of Sound

1. **Frequency (Hz)** – Number of vibrations per second.
    - High frequency → High pitch sound
    - Low frequency → Low pitch sound
2. **Amplitude** – Determines the loudness of the sound.
3. **Waveform** – Shape of the signal that generates sound.

Arduino generates sound by producing **square wave signals** on a digital pin.

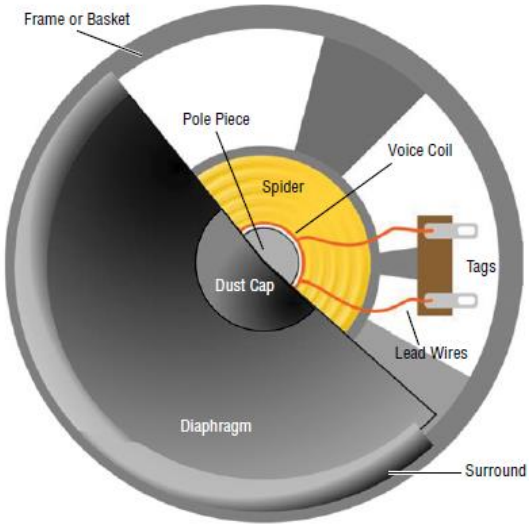
HOW A SPEAKER PRODUCES SOUND

Speakers are electro-mechanical devices that convert **electrical signals into sound waves**. They work based on the principle of **electromagnetism**, which allows electricity to produce mechanical motion. When an electrical signal from a device such as an Arduino, amplifier, or audio source is applied to a speaker, the speaker vibrates and generates sound waves in the air.

In Arduino projects, speakers are often used with functions such as **tone( )** to generate sound signals. These electrical signals cause the internal parts of the speaker to move, producing audible sound.

A speaker operates using electromagnetic forces. When current flows through a wire coil placed in a magnetic field, a force is produced. This force causes the coil and attached diaphragm to move back and forth, generating sound waves.

The continuous movement of the diaphragm pushes air forward and backward, creating pressure waves that travel through the air and reach our ears as sound.



Speaker cross section

Making Sound Using tone ( ) Function

Arduino provides the tone() function to generate a square wave at a specified frequency.

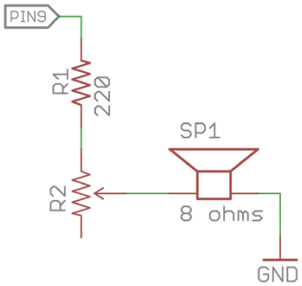
Syntax

```
tone(pin, frequency, duration);
```

Stopping the Sound Syntax

```
noTone(pin);
```

Wiring the Speaker



Components Required

- Arduino board
- Piezo speaker or buzzer

- 100–220 Ω resistor (optional)
- Breadboard
- Connecting wires

Program to produces tone of **1000 Hz for one second**.

```
int speakerPin = 9;

void setup( )
{
}

void loop( )
{
  tone(speakerPin, 1000); // Generate 1000 Hz tone
  delay(1000);          // Play for 1 second
  noTone(speakerPin);     // Stop sound
  delay(1000);
}
```

- tone( ) generates a square wave signal.
- delay( ) controls the duration.
- noTone( ) stops the sound.

Program to Generate Sound Sequence

```
#include "pitches.h"     // Header file with pitch definitions

const int SPEAKER = 9;   // Speaker pin

// Note array
int notes[] = {
  NOTE_A4, NOTE_E3, NOTE_A4, 0,
  NOTE_A4, NOTE_E3, NOTE_A4, 0,
  NOTE_E4, NOTE_D4, NOTE_C4, NOTE_B4, NOTE_A4, NOTE_B4, NOTE_C4,
NOTE_D4,
  NOTE_E4, NOTE_E3, NOTE_A4, 0
};

// Duration of each note (in ms)
int times[] = {
  250, 250, 250, 250,
```

```
  250, 250, 250, 250,
  125, 125, 125, 125, 125, 125, 125, 125,
  250, 250, 250, 250
};

int numNotes = sizeof(notes) / sizeof(notes[0]);

void setup()
{
  for (int i = 0; i < numNotes; i++)
  {
    tone(SPEAKER, notes[i], times[i]); // Play note
    delay(times[i] * 1.3);           // Wait for note duration
    noTone(SPEAKER);                 // Stop tone
  }
}

void loop()
{
  // Press reset button to play again
}
```

Limitations of the tone( ) Function

Although tone( ) is simple and useful, it has some limitations:

- Only **one tone can be generated at a time**.
- It uses **hardware timers**, which may conflict with other libraries.
- Sound quality is limited because it generates only **square waves**

INTRODUCTION TO INTERRUPTS

Until now, most Arduino programs execute instructions synchronously, meaning that the program executes one command after another inside the loop() function. While this approach works for simple programs, it can create problems when the program contains delays or time-consuming operations.

For example, if an Arduino program is executing a long delay() or running a loop that gradually increases the brightness of an LED, the processor cannot respond to external events such as a button press during that time. As a result, important inputs may be missed.

To overcome this limitation, Arduino supports interrupts, which allow the microcontroller to respond to events immediately, regardless of what the main program is doing. Interrupts enable the program to execute asynchronously, meaning that certain functions can run automatically when specific events occur.

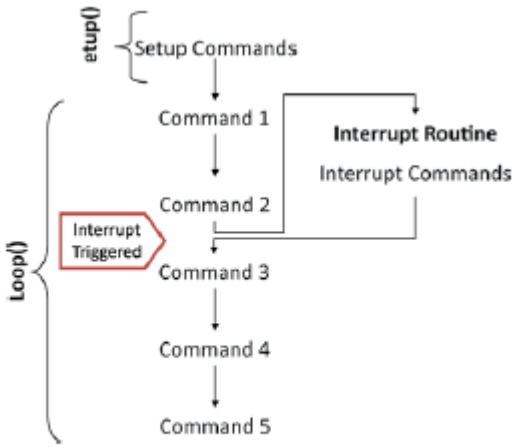
When an interrupt occurs, the Arduino temporarily pauses the current task, executes a special function called an Interrupt Service Routine (ISR), and then returns to the original program execution.

Hardware Interrupts

Hardware interrupts are generated by **external signals connected to specific input pins** of the microcontroller. When the state of one of these pins changes, the Arduino hardware detects the change and automatically triggers an interrupt.

Hardware interrupts are useful when an event must be detected immediately without continuously checking the state of the input pin.

For instance, suppose an Arduino is controlling the speed of a motor while gradually increasing its brightness using a loop with delays. If a button press is used to change the motor speed or LED color, the program may miss the button press if it occurs during the delay period. Hardware interrupts solve this issue by allowing the Arduino to detect the button press instantly, even while the main program is executing other instructions.



How an external interrupt affects program flow

Thus, hardware interrupts allow the microcontroller to respond to external events **at any moment during program execution**.

Polling Method

When designing embedded systems, developers often need to choose between **polling inputs** and **using interrupts**. Both approaches have advantages and limitations.

Polling means repeatedly checking the status of an input device inside the program loop using a function such as digitalRead( ).

For example, a program may repeatedly check whether a button is pressed during each iteration of the loop.

TRADE-OFFS BETWEEN POLLING AND INTERRUPTING

Hardware interrupts provide an alternative method to repeatedly **polling external inputs** inside the loop() function of an Arduino program. Interrupts are not necessarily better or worse than polling; instead, each method has its own advantages and limitations. When designing an embedded system, the programmer must evaluate the requirements of the application and decide whether **polling or interrupts** are more appropriate.

Polling involves continuously checking the state of an input device (such as a push button) inside the program loop. Interrupts, on the other hand, allow the microcontroller to automatically respond to changes in input signals without continuously checking them.

❖ ***Ease of Implementation (Software)***

From a software perspective, implementing **polling** is generally simpler than using interrupts.

In polling, the programmer only needs to check the state of an input using a simple function such as:

```
digitalRead(pin);
```

This method is easy to understand and implement because the input is checked at regular intervals within the program loop.

Although Arduino provides convenient functions to attach interrupts, using interrupts still requires writing additional code. For example, the programmer must define an **interrupt service routine (ISR)** and attach it to a specific interrupt pin using the attachInterrupt() function.

Therefore, if an application does not require the advanced capabilities of interrupts, polling is often the simpler option.

❖ ***Ease of Implementation (Hardware)***

For most digital input circuits, the hardware configuration for **polling and interrupting is identical**. In both cases, the microcontroller simply detects a change in the voltage level of the input signal.

However, a special consideration arises when using **edge-triggered interrupts**, especially with mechanical push buttons.

Mechanical switches tend to **bounce** when pressed. Instead of producing a single clean transition from HIGH to LOW, the signal rapidly oscillates several times before settling to its final state. This

bouncing can cause the interrupt routine to be triggered multiple times, even though the user intended to press the button only once.

Another complication is that traditional **software debouncing techniques cannot be used inside interrupt routines**, because interrupt functions cannot contain delays such as delay( ).

Therefore, when using interrupts with mechanical buttons, it is often necessary to implement **hardware debouncing circuits** before connecting the signal to the interrupt pin.

If the input signal does not bounce, such as signals generated by:

- rotary encoders
- digital outputs from integrated circuits

then the hardware required for interrupting is the same as that used for polling.

- ***Multitasking***

One of the main advantages of interrupts is their ability to enable **pseudo-multitasking**.

An Arduino cannot perform true multitasking because it contains only **one microcontroller unit (MCU)** that can execute only one instruction at a time. However, the processor operates at a very high speed, allowing it to switch between tasks rapidly.

Interrupts make it possible to **interleave multiple operations**, giving the appearance that several tasks are running simultaneously.

For example, consider a program that gradually changes the brightness of an LED using a loop with delays. If the system also needs to detect a button press to adjust the LED color or brightness speed, polling may fail because the program checks the button state only after finishing the delay.

Using interrupts, the system can:

- continue running the LED fading routine
- immediately respond to a button press

This creates the effect that both tasks are running at the same time.

- ***Acquisition Accuracy***

For certain applications that require **precise timing or fast signal detection**, interrupts are essential.

Consider the example of a **rotary encoder** connected to a DC motor. A rotary encoder generates a pulse each time the motor rotates by a specific amount. These pulses are used to measure the motor's position and speed.

Because these pulses are very short in duration, they may be missed if the program only checks the signal occasionally using polling. Missing even a single pulse can cause significant errors in measurements. For instance, if an encoder generates one pulse per revolution and the program misses one pulse, the system may incorrectly assume that the motor speed is half of its actual value.

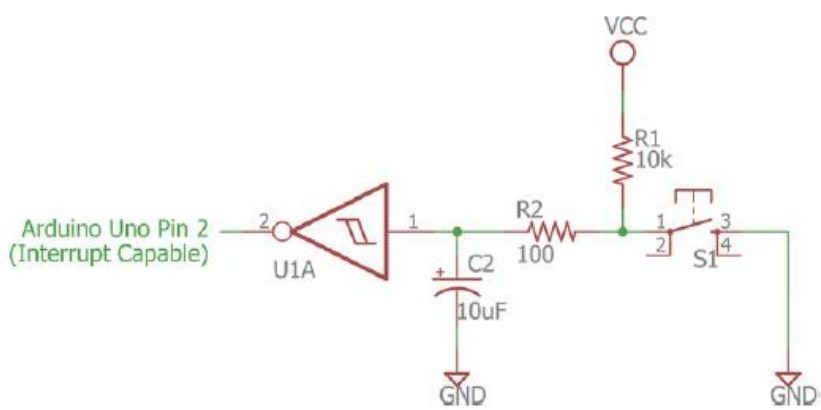
Interrupts ensure that every pulse is captured immediately, making them ideal for applications that require **high acquisition accuracy**.

In contrast, for slowly changing inputs such as a push button pressed by a human user, polling may still be sufficient.

DEBOUNCE CIRCUIT USING AN INVERTING SCHMITT TRIGGER

A debounce circuit is used to eliminate the unwanted multiple signals produced when a mechanical switch is pressed or released. Mechanical switches do not change state instantly from open to closed. Instead, due to the physical nature of the metal contacts, the switch contacts bounce several times before settling to a stable state. This bouncing creates multiple rapid transitions between HIGH and LOW levels within a few milliseconds. When such a signal is connected directly to a digital system or microcontroller input, the system may interpret these multiple transitions as several button presses instead of a single press. To avoid this problem, hardware debouncing circuits are used to produce a clean and stable digital signal.

One effective method of hardware debouncing uses an **RC filter combined with an inverting Schmitt trigger**. The RC network smooths the rapid fluctuations caused by switch bounce, while the Schmitt trigger converts the slowly changing analog signal into a clean digital output. This approach ensures reliable switching and is particularly useful when the switch is connected to interrupt pins of microcontrollers such as the Arduino.



Debounce Circuit Using an Inverting Schmitt Trigger

In the circuit, a push button switch is connected to ground, while a pull-up resistor connects the input node to the supply voltage. When the switch is not pressed, the pull-up resistor keeps the node at a HIGH voltage level. A capacitor is connected between the input node and ground. This capacitor forms an RC network with the resistor and helps filter out rapid voltage changes. The filtered signal is then applied to the input of an **inverting Schmitt trigger gate**, which provides a clean digital output. The output of the Schmitt trigger is connected to the microcontroller input pin.

When the push button is in the open state, the pull-up resistor pulls the voltage at the input node toward the supply voltage. The capacitor charges through the resistor and maintains the voltage at a stable HIGH level. Since the signal is applied to an inverting Schmitt trigger, the output of the gate becomes LOW. The microcontroller therefore detects a stable LOW logic level at its input pin when the switch is not pressed.

When the switch is pressed, the input node is connected to ground. As a result, the capacitor begins to discharge gradually through the circuit. Because the capacitor discharges slowly, the voltage does not drop instantly but instead decreases smoothly. The Schmitt trigger monitors this changing voltage. Once the voltage falls below the lower threshold voltage of the Schmitt trigger, the gate switches its output state. Because the gate is an inverter, the output becomes HIGH, and the microcontroller detects a logic HIGH signal indicating that the button has been pressed.

During the bouncing period of the mechanical switch, the RC network prevents rapid voltage fluctuations from appearing directly at the input of the Schmitt trigger. The capacitor absorbs short pulses caused by bouncing and ensures that the voltage changes gradually rather than abruptly. Additionally, the Schmitt trigger provides hysteresis, which means it uses two different switching thresholds for rising and falling voltages. This hysteresis prevents small variations in voltage from repeatedly switching the output, thereby producing a stable digital signal.

The combination of the RC filter and the Schmitt trigger makes this hardware debouncing method very reliable. It eliminates false triggering, ensures clean digital transitions, and improves the stability of systems that rely on push button inputs. This circuit is widely used in microcontroller applications, digital systems, control panels, and embedded systems where accurate detection of switch presses is essential. By providing a clean signal to the microcontroller, the debounce circuit ensures that each physical press of the button results in exactly one digital event.

Hardware-Debounced Switch to Control an Interrupt-Example code

When a hardware debounce circuit (RC network + Schmitt trigger) is used with a push button, the output signal becomes a clean digital pulse. This clean signal can be safely connected to an interrupt pin of the Arduino so that the microcontroller responds immediately whenever the

button is pressed. Since the hardware circuit removes the switch bounce, the interrupt will be triggered only once for each button press, eliminating the need for software debouncing.

In the Arduino Uno, digital pins 2 and 3 support external interrupts. The output of the Schmitt trigger debounce circuit is usually connected to Pin 2. When the button is pressed, the signal changes state (LOW to HIGH or HIGH to LOW depending on the circuit configuration), and the Arduino detects this change and executes an interrupt service routine (ISR).

In this program, an interrupt is attached to Pin 2 using the attachInterrupt() function. When the interrupt occurs, the Arduino calls a function that performs the required action. The interrupt routine should always be short and simple because it temporarily pauses the main program execution.

```
const int buttonPin = 2;   // Interrupt pin connected to debounced switch
const int ledPin = 13;     // Built-in LED

volatile bool buttonState = false;

void setup( )
{
  pinMode(buttonPin, INPUT);
  pinMode(ledPin, OUTPUT);

  Serial.begin(9600);

  // Attach interrupt to pin 2
  attachInterrupt(digitalPinToInterrupt(buttonPin), buttonPressed, RISING);
}

void loop( )
{
  if(buttonState == true)
  {
    Serial.println("Button Press Detected");
    digitalWrite(ledPin, HIGH);
    delay(500);
    digitalWrite(ledPin, LOW);
    buttonState = false;
  }
}

void buttonPressed( )
{
  buttonState = true;   // Interrupt Service Routine
}
```

The program begins by defining Pin 2 as the interrupt input pin connected to the debounced switch and Pin 13 as the LED output pin. A volatile boolean variable is used to store the interrupt event because variables shared between the interrupt routine and the main program must be declared as volatile.

Inside the setup( ) function, the pin modes are configured and serial communication is started for displaying messages on the serial monitor. The attachInterrupt( ) function is then used to connect the interrupt pin with the interrupt service routine. The argument digitalPinToInterrupt(buttonPin) identifies the interrupt associated with pin 2, and RISING indicates that the interrupt will occur when the signal transitions from LOW to HIGH.

Whenever the hardware-debounced switch is pressed, the signal from the Schmitt trigger changes state and triggers the interrupt. The Arduino immediately executes the function buttonPressed( ), which sets the buttonState variable to true.

In the main loop( ) function, the program checks the value of buttonState. If it is true, the program prints a message on the serial monitor and briefly turns on the LED connected to pin 13. After the action is completed, the variable is reset so that the system can detect the next button press.

TIMER INTERRUPTS IN ARDUINO

A timer interrupt is a type of interrupt generated by a hardware timer inside the microcontroller. Unlike hardware interrupts that are triggered by external events such as button presses, timer interrupts occur at regular time intervals determined by the internal timer. These interrupts allow a microcontroller to perform specific tasks periodically without constantly checking the time in the main program loop. Timer interrupts are widely used in embedded systems for tasks such as LED blinking, signal generation, motor control, data sampling, and real-time scheduling.

In Arduino boards based on the **ATmega328P microcontroller**, such as the Arduino Uno, there are **three hardware timers** available. These timers can be used for many timing-related operations including generating PWM signals, measuring time intervals, and triggering interrupts. The Arduino environment already uses these timers internally to implement functions such as millis(), delay(), and analogWrite(). However, programmers can also take manual control of the timers to create precise periodic events.

Working of Timer Interrupts

A hardware timer works similarly to a stopwatch. It begins counting from zero and increments its value with every clock cycle generated by the oscillator inside the microcontroller. The ATmega328P used in Arduino Uno operates at a **16 MHz clock frequency**, meaning the timer can count extremely fast. Timer1 in the ATmega328P is a **16-bit timer**, which means it can count from **0 to 65,535** ($2^{16} - 1$). Once the maximum value is reached, the timer resets back to zero and begins counting again.

The speed at which the timer reaches its maximum count depends on the clock frequency and the clock divider (also called prescaler). By dividing the clock signal, the timer can count more slowly, allowing longer timing intervals. Instead of manually calculating these values, programmers often use libraries that simplify timer configuration.

One commonly used library for controlling Timer1 in Arduino is the **TimerOne library**. This library simplifies the use of timer interrupts by allowing the programmer to directly specify the time interval in microseconds. When the specified interval elapses, the Arduino automatically executes a predefined function known as the **interrupt service routine (ISR)**.

Installing the TimerOne Library

To use timer interrupts easily, the **TimerOne library** must be installed in the Arduino IDE. This can be done through the Library Manager in the Arduino environment. After opening the Arduino IDE, navigate to **Sketch → Include Library → Manage Libraries** and search for the TimerOne library. Once installed, the library allows direct control of the 16-bit Timer1 module of the ATmega328P microcontroller.

It should be noted that Timer1 is also used internally by Arduino for generating PWM signals on **pins 9 and 10**. When the TimerOne library is used, PWM output through analogWrite() on these pins cannot be used simultaneously.

Timer Interrupt Example: LED Blink Using Timer1

A simple example of a timer interrupt is blinking the onboard LED connected to **pin 13** of the Arduino Uno. In this example, the timer generates an interrupt every one second. Each time the interrupt occurs, a function is executed that toggles the state of the LED.

```
// Using Timer Interrupts with Arduino
#include <TimerOne.h>

const int LED = 13;

void setup()
{
  pinMode(LED, OUTPUT);

  // Initialize Timer1 with a period of 1 second
  Timer1.initialize(1000000);

  // Attach interrupt function
  Timer1.attachInterrupt(blinky);
}

void loop()
```

```
{
  // Other code can run here simultaneously
}

// Timer interrupt function
void blinky()
{
  digitalWrite(LED, !digitalRead(LED)); // Toggle LED state
}
```

The program begins by including the **TimerOne library**, which allows easy control of Timer1. A constant variable is declared to represent the LED connected to pin 13. Inside the setup() function, the LED pin is configured as an output.

The function Timer1.initialize(1000000) sets the timer period to **1,000,000 microseconds**, which is equal to **1 second**. This means the timer interrupt will occur once every second. The statement Timer1.attachInterrupt(blinky) attaches a function named blinky() to the timer interrupt. Every time the timer interval expires, the Arduino automatically calls this function.

The loop() function remains empty in this example, but other tasks could be executed here. The timer interrupt will still occur in the background, giving the appearance that multiple operations are happening simultaneously.

The function blinky() acts as the **interrupt service routine**. This function toggles the state of the LED by writing the opposite of its current value. As a result, the LED turns on and off every second.

Advantages of Timer Interrupts

Timer interrupts allow tasks to be executed at precise time intervals without continuously checking the system clock. They improve program efficiency because the microcontroller can perform other operations in the main loop while waiting for the timer event. Timer interrupts are especially useful in real-time systems where periodic operations such as sensor sampling, waveform generation, and communication timing must occur accurately.